Cartella Principale: Firpo_Thesis_Project

Al livello radice della cartella sono presenti i documenti generali del lavoro di tesi:

- Presentazione di tesi (file PowerPoint utilizzato in sede di discussione).
- PDF della tesi (documento di tesi completo).
- Cartella ASG_Application
- Cartella MachineLearning_&_DeepLearning

Cartella ASG_Application

Questa sezione raccoglie tutto il materiale utilizzato per testare e applicare i due modelli (Machine Learning e Deep Learning) su immagini di pre- e post-taratura, con l'obiettivo di verificare lo shift delle immagini dalla classe 0 (pre-taratura) verso le classi 2/3 (post-taratura).

- **Dataset:** Cartelle contenenti le immagini divise in pre-taratura e post-taratura.
- **Risultati:** Output di classificazione ottenuti applicando entrambi i modelli:
    - **Random Forest** (Machine Learning)
    - **ResNet-18** (Deep Learning)
- **Codici Python di riferimento:**
    - validation_CNN_fromFolder.py
    - validation_RF_fromFolder.py

Cartella Machine_Learning_&_DeepLearning

Questa è la cartella principale di sviluppo e contiene l'intero flusso di lavoro suddiviso in 4 sottocartelle (A,B,C e D)

A. Sottocartella dataset

Contiene i dati grezzi e i riferimenti utilizzati per l'addestramento e il test:

- images: Tutte le immagini originali del dataset.
- DicomForValidation: 8 immagini dedicate a una prima e semplice validazione dei modelli.
- classification.xlsx: File Excel con la classificazione manuale originale fornita da ASG Superconductors S.p.A., usata come riferimento e ground truth per il training.
- ml_test_ids.xlsx: File Excel utilizzato per garantire l'utilizzo dello stesso identico test set sia per il Machine Learning che per il Deep Learning.
- *Nota sui file non utilizzati:* All'interno della cartella sono presenti anche consensus.xlsx, Analysis_Simona_Data.xlsx e Consensus_Analisi_Media.xlsx, i quali non sono stati impiegati né richiamati nei codici.

B. Sottocartella Machine_Learning

Raccoglie lo sviluppo e l'implementazione dei modelli di Machine Learning:

- **Progressive_Steps**: Contiene 4 sottocartelle relative alle principali prove intermedie effettuate. Ognuna include:
    - Il codice Python dedicato.
    - I risultati delle metriche del modello (metriche_modello con relativa matrice di confusione).
    - Le predizioni sul test set (test_predictions).
- **Results_ML_Firpo_+Val**: Raccolta del modello di Machine Learning finale basato sull'estrazione manuale di feature (GLCM, Sobel e FFT). Al suo interno troviamo:
    - ML_Code.py: Codice Python di riferimento.
    - Modello addestrato pronto all'uso in formato .pkl.
    - Risultati sul test set (metriche_modello con matrice di confusione e test_predictions).
    - validation_firpo.py: Codice Python utilizzato per la validazione sulle 8 immagini DICOM (DicomForValidation).
    - validation_risultati.xlsx: Risultati della validazione suddetta.

C. Sottocartella Deep_Learning

Raccoglie l'implementazione e i risultati delle reti neurali:

- npyCOP1GEMNOGRAD: Immagini del dataset convertite in formato .npy.
- **results_forced_test_60epochs_cnn_cop1_gemOptim_docker**:
    - forced_test_60_epo_cnn_COP1_Gem_optim.py: Codice Python principale utilizzo CNN.
    - resnet_mri_artifacts.pth: Pesi della rete addestrata, utilizzabile per classificare nuove immagini (approccio analogo a quanto fatto in ASG_Application).
    - File Excel dei risultati sul test set forzato (coincidente con quello del ML): classification_report, confusion_matrix e test_predictions, utili per valutare l'affidabilità e l'accuratezza nel riconoscimento degli artefatti gravi e meno gravi.
    - visualize_CNN_Feature_NoGrid.py: Codice per l'estrazione delle feature visive della rete neurale (le immagini estratte sono salvate in individual_plots).
    - valid_CNN.py e Validation_CNN_Forced.xlsx: Codice e risultati Excel relativi alla validazione sulle 8 immagini di DicomForValidation tramite il file .pth.

D. Sottocartella test_statistico_RC2

Dedicata all'analisi statistica dei risultati:

- Documento PDF con le specifiche e le indicazioni ricevute per l'esecuzione dei test.
- test_statistico_RC.py: Script Python che esegue i due test statistici sulle metriche di *Accuracy* e *QWK* (Quadratic Weighted Kappa).
- Risultati generati:
    - File Excel con la tabella di confronto.
    - I 3 plot grafici generati.
    - Le due matrici di confusione.